SIMATS Engineering • CSE • CSA0804 — Python Programming • 29-07-2026 • Category: Class Work (28.7.26) • Student Submissions Report

Student 25 **Perarasan V** 192521404RD 19 / 20 submitted

Q1. Void Function (No Return Value)

CODE

```
def welcome():
    print("Welcome to Python Programming")

welcome()
```

INPUT

```
(none)
```

OUTPUT

```
Welcome to Python Programming
```

Q2. Void Function (No Return Value)

CODE

```
def multiplication_table(n):
    for i in range(1, 11):
        print(f"{n} x {i} = {n * i}")

n = int(input())
multiplication_table(n)
```

INPUT

```
6
```

OUTPUT

```
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
```

Q3. Void Function (No Return Value)

CODE

```
def print_even(n):
    for i in range(1, n + 1):
        if i % 2 == 0:
            print(i)

n = int(input())
print_even(n)
```

INPUT

```
10
```

OUTPUT

```
2
4
6
8
10
```

Q4. Fruitful Function (Returns One Value)

CODE

```
def count_vowels(s):
    count = 0
    for char in s:
        if char.lower() in 'aeiou':
            count += 1
    return count

s = input()
print(count_vowels(s))
```

INPUT

```
Programming
```

OUTPUT

```
3
```

Q5. Fruitful Function (Returns One Value)

CODE

```
def largest_element(nums):
    return max(nums)

lst = [1, 5, 3, 9, 2]
print(largest_element(lst))
```

INPUT

```
1 5 3 9 2
```

OUTPUT

```
9
```

Q6. Fruitful Function (Returns One Value)

CODE

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True


n = int(input())
print("True" if is_prime(n) else "False")
```

INPUT	5
OUTPUT	True

Q7. Function Returning Two or More Values

CODE

```
def marks_result(sub1, sub2, sub3):
    total = sub1 + sub2 + sub3
    avg = total / 3

    if avg ≥ 90:
        grade = "A"
    elif avg ≥ 80:
        grade = "B"
    elif avg ≥ 70:
        grade = "C"
    elif avg ≥ 60:
        grade = "D"
    else:
        grade = "F"

    return total, avg, grade

t, a, g = marks_result(85, 92, 78)
print("Total:", t)
print("Average:", a)
print("Grade:", g)
```

INPUT

```
85
92
78
```

OUTPUT

```
Total: 255
Average: 85.0
Grade: B
```

Q8. Function Returning Two or More Values

CODE

```
import math

def circle_area_circumference(radius):
    area = math.pi * radius * radius
    circumference = 2 * math.pi * radius
    return area, circumference

r = float(input())
area, circumference = circle_area_circumference(r)

print("Area:", area)
print("Circumference:", circumference)
```

INPUT

```
3
```

OUTPUT

```
Area: 28.274333882308138
Circumference: 18.84955592153876
```

Q9. Function Returning Two or More Values

CODE

```
import math

def square_cube_sqrt(n):
    square = n ** 2
    cube = n ** 3
    sqrt = math.sqrt(n)
    return square, cube, sqrt

x = float(input())
s, c, r = square_cube_sqrt(x)
print(s, c, r)
```

INPUT

```
4
```

OUTPUT

```
16.0 64.0 2.0
```

Q10. Keyword Arguments

CODE

```
def student(name, age, department):
    print("Name:", name)
    print("Age:", age)
    print("Department:", department)

student(name="Perarasan V", age=18, department="B.Tech IT")
```

OUTPUT

```
Name: Perarasan V
Age: 18
Department: B.Tech IT
```

Q11. Keyword Arguments

CODE

```
def employee(empid, name, salary):
    print("Empid:", empid)
    print("Name:", name)
    print("Salary:", salary)

employee(salary=45000, name="Perarasan V", empid=101)
```

INPUT

```
empid=101, name=Perarasan V, salary=45000
```

OUTPUT

```
Empid: 101
Name: Perarasan V
Salary: 45000
```

Q12. Keyword Arguments

CODE

```
def book(title, author, price):
    print("Book Details:")
    print("Title :", title)
    print("Author:", author)
    print("Price :", price)

book(title="The Alchemist", author="Paulo Coelho", price=499)
```

INPUT

```
title = "The Alchemist", author = "Paulo Coelho", price = 499
```

OUTPUT

```
Book Details:
Title : The Alchemist
Author: Paulo Coelho
Price : 499
```

Q13. Variable-Length Arguments (*args)

CODE

```
def add_numbers(*args):
    return sum(args)

print(add_numbers(1, 2, 3))
```

INPUT
```
1,2,3
```

OUTPUT
```
6
```

Q14. Variable-Length Arguments (*args)

CODE

```
def average_marks(*marks):
    if len(marks) == 0:
        return 0
    return sum(marks) / len(marks)

print(average_marks(50, 60, 70, 80))
```

INPUT
```
50 60 70 80
```

OUTPUT
```
65.0
```

Q15. Variable-Length Arguments (*args)

CODE

```
def print_strings(*args):
    for s in args:
        print(s)

print_strings("Hello", "World", "Python")
```

OUTPUT

```
Hello
World
Python
```

Q16. Variable-Length Arguments (*args)

CODE

```
def largest_number(*args):
    return max(args)

print(largest_number(3, 7, 2, 10, 5))
```

INPUT

```
3 7 2 10 5
```

OUTPUT

```
10
```

Q17. Recursive Functions

CODE

```
def fibonacci_series(n):
    if n <= 0:
        return
    def fib(i):
        if i == 0:
            return 0
        if i == 1:
            return 1
        return fib(i-1) + fib(i-2)

    for i in range(n):
        print(fib(i), end=" " if i != n-1 else "")

n = int(input())
fibonacci_series(n)
```

INPUT	5
OUTPUT	0 1 1 2 3

Q18. Recursive Functions

CODE

```
def sum_of_digits(n):
    n = abs(n)
    if n < 10:
        return n
    return (n % 10) + sum_of_digits(n // 10)
```

```
num = int(input("Enter a number: "))
print(sum_of_digits(num))
```

INPUT

```
12345
```

OUTPUT

```
Enter a number: 15
```

Q19. Recursive Functions

CODE

```
def reverse_string(s):

    if len(s) ≤ 1:
        return s
    return reverse_string(s[1:]) + s[0]

s = input("Enter a string: ")
print(reverse_string(s))
```

INPUT

```
hello
```

OUTPUT

```
Enter a string: olleh
```

Q20. Recursive Functions

— Not Submitted —